

## Article

# Open-Source Real-Time SDR Platform for Rapid Prototyping of LANS AFS Receiver

Rion Sobukawa and Takuji Ebinuma \*

Department of Astronautics and Aeronautics, Chubu University, Kasugai 487-8501, Aichi, Japan

\* Correspondence: [ebinuma@fsc.chubu.ac.jp](mailto:ebinuma@fsc.chubu.ac.jp)

## Abstract

The Lunar Augmented Navigation Service (LANS) is the lunar equivalent of GNSS for future lunar explorations. It offers users accurate position, navigation, and timing (PNT) capabilities on and around the Moon. The Augmented Forward Signal (AFS) is a standardized signal structure for LANS, and its recommended standard was published online on 7 February 2025. This work presents software-defined radio (SDR) implementations of the LANS AFS simulator and receiver, which were rapidly developed within a month of the signal specification release. Based on open-source GNSS software, including GPS-SDR-SIM and Pocket SDR, our system provides a valuable platform for future algorithm research and hardware-in-the-loop testing. The receiver can operate on embedded platforms, such as the Raspberry Pi 5, in real-time. This feature makes it suitable for lunar surface applications, where conventional PC-based SDR systems are impractical due to their size, weight, and power requirements. Our approach demonstrates how open-source SDR frameworks can be rapidly applied to emerging satellite navigation signals, even for extraterrestrial PNT applications.

**Keywords:** LunaNet; Lunar Augmented Navigation Service (LANS); Augmented Forward Signal (AFS); Software Defined Radio (SDR)



Academic Editor: M. Reza Emami

Received: 29 May 2025

Revised: 8 July 2025

Accepted: 8 July 2025

Published: 10 July 2025

**Citation:** Sobukawa, R.; Ebinuma, T. Open-Source Real-Time SDR Platform for Rapid Prototyping of LANS AFS Receiver. *Aerospace* **2025**, *12*, 620. <https://doi.org/10.3390/aerospace12070620>

**Copyright:** © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

## 1. Introduction

LunaNet is a proposed lunar communication and navigation network designed to provide robust, interoperable, scalable services for exploration missions [1]. Developed by an international partnership of NASA, ESA, and JAXA, LunaNet aims to establish a framework similar to the terrestrial Internet and GNSS, enabling high-data-rate communications, precise positioning, and the relay of scientific data across the Moon.

The Lunar Augmented Navigation Service (LANS), a key component of LunaNet, is designed to provide scalable and interoperable positioning, navigation, and timing (PNT) services for lunar operations. The Augmented Forward Signal (AFS) is a standardized navigation signal structure for LANS. It is designed to offer users accurate positioning and timing information on and around the Moon. The LANS supports a GNSS-like service by broadcasting the signals from multiple LunaNet Service Provider (LNSP) satellites orbiting the Moon.

Unlike GNSS signals, which can be observed anywhere on Earth, no broadcast LANS AFS is available. Therefore, a dedicated signal simulator is required to generate LANS AFS waveforms. Simultaneously, a LANS AFS reference receiver is needed to validate the generated waveforms. For these reasons, the receiver and the signal simulator must be developed concurrently.

The LunaNet Signal-In-Space Recommended Standard for AFS (LSIS-AFS) was published online on 7 February 2025 [2] as part of the applicable documents for LunaNet Interoperability Specification (LNIS) Version 5 [3]. This document defines the specifications of the modulation scheme and data frame structure of LANS AFS. However, some essential properties for interoperable PNT services remain undefined. In particular, the definitions of time and coordinate systems are currently under discussion, and the data format of the corresponding navigation message is undefined [4,5]. Therefore, the receiver and signal simulator platform must be flexible to accommodate future specification updates. To achieve this flexibility, we utilize software-defined radio (SDR) in this study.

GNSS researchers widely use SDR technology to test new algorithms and prototype highly customized receivers [6]. However, most existing software-based GNSS receivers require a desktop or laptop computer to operate, and only a few are open-source. Pocket SDR [7], developed and released by Tomoji Takasu, is the only open-source project we know of that has proven to operate efficiently on embedded platforms such as the Raspberry Pi 5. Because of the similarities between AFS and conventional GNSS signal structures, we leverage the flexibility and open-source nature of Pocket SDR to develop a lightweight and power-efficient LANS AFS receiver platform. This implementation can realize practical navigation receivers for lunar missions, where size, weight, and power (SWaP) constraints are critical.

In parallel, a signal simulator that generates LANS AFS waveforms is required to validate the signal processing algorithm and assess the navigation performance of the receiver. GPS-SDR-SIM [8], an open-source GPS baseband signal simulator developed by one of the authors, served as the base platform for this task. Leveraging its architecture, we created the LANS AFS simulator, which generates a simulated baseband signal that can be directly processed by SDR receiver software or transmitted as an RF signal via an SDR device. Following LSIS-AFS recommendations, we have implemented all necessary functions for generating ranging codes and constructing navigation message frames. However, the contents of the Clock and Ephemeris Data (CED) message remain undefined in the current recommendations. Without this data, the receiver cannot determine its position. The current implementation assumes a spherical Moon and simple Keplerian orbits for the LNSP satellites. Thanks to the flexibility of SDR, these assumptions can be readily updated once the official CED message format is released.

This paper provides an overview of our recent achievements in developing a complete SDR-based LANS AFS simulator and a receiver. Leveraging open-source GNSS tools, such as GPS-SDR-SIM and Pocket SDR, we successfully prototyped and validated a functional LANS AFS receiver within one month of the signal specification release. Our approach demonstrates how modular and open-source SDR frameworks can be rapidly adapted to emerging satellite navigation signals, even for extraterrestrial PNT applications. The resulting system supports end-to-end simulation and reception of LANS AFS, providing a valuable platform for future algorithm research and hardware-in-the-loop testing.

In summary, the contributions of this work are threefold: (1) the first SDR-based implementation of a functional LANS AFS receiver operating in real time on embedded hardware; (2) an efficient LANS AFS baseband signal generator optimized with time-delay approximation and multi-core parallelization; and (3) a demonstration of how SDR-based approaches enable rapid adaptation to emerging navigation signals, overcoming limitations inherent in traditional hardware implementation.

The remainder of this paper is organized as follows: Section 2 outlines the LANS AFS specifications and system design. Section 3 describes the baseband signal generation of LANS AFS. The techniques enabling efficient signal generation, including time-delay approximation and multi-core parallelization, are presented in this section. Section 4

provides a detailed description of the SDR-based LANS AFS receiver implementation. Section 5 validates the complete system using both offline and real-time tests on embedded hardware, demonstrating the practical feasibility and performance of the proposed platform. Finally, Section 6 discusses limitations and outlines directions for future enhancements.

## 2. LANS AFS Overview

One of the distinct differences between the LANS AFS and the conventional GNSS signals is the center frequency. Following the SFCG REC 32-2R6 recommendation from the Space Frequency Coordination Group [9], the LANS AFS is allocated in the S-band with a center frequency of 2492.028 MHz, while the conventional GNSS signals operate in the L-band.

The LANS AFS employs a modern dual-channel architecture consisting of Data and Pilot channels. The Data channel carries navigation messages containing satellite orbit and clock information. On the other hand, the Pilot channel without data modulation is designed for robust signal acquisition and tracking through long coherent integration. These channels are quadrature-multiplexed onto the in-phase (I) and quadrature (Q) components of the carrier wave, a similar structure found in modernized GNSS signals, such as GPS L1C [10].

Both channels utilize binary phase shift keying (BPSK) modulation with distinct pseudorandom noise (PRN) spreading codes. The Data channel employs a 2046-chip Gold code derived by short-cycling a 2047-length sequence. With a 1.023 Mcps spreading rate, equivalent to the GPS L1 C/A signal, the code period of the Data channel becomes 2 ms. The navigation message is encoded using a rate-1/2 low-density parity check (LDPC) code for forward error correction (FEC), with interleaving to enhance resilience against burst errors, and transmitted at a symbol rate of 500 symbols per second (sps).

The Pilot channel utilizes a 10,230-chip PRN code, identical to the GPS L1C primary code, resulting in a 5.115 Mcps spreading rate while maintaining the same 2 ms code period as the Data channel. It carries no navigation message but incorporates a hierarchical structure of secondary and tertiary codes layered on the primary spreading code. This design supports extended coherent integration during acquisition and improves phase-tracking performance under weak-signal conditions.

More detailed features of the LANS AFS data frames and overlay codes, including a robust frame synchronization structure, can be found in Reference [11].

Table 1 provides a comparative summary of the LANS AFS characteristics relative to those of conventional GNSS signals. It should be noted that, unlike LANS AFS and GPS L1C, which transmit the Data and Pilot signals on orthogonal I and Q channels, the Galileo E1 signal transmits both the E1-B (Data) and E1-C (Pilot) components on the in-phase channel only, using a Composite Binary Offset Carrier (CBOC) modulation scheme [12].

**Table 1.** Key signal features of LANS AFS compared with conventional GNSS signals.

| Feature                | LANS AFS    | GPS L1 C/A | GPS L1C   | Galileo E1    |
|------------------------|-------------|------------|-----------|---------------|
| Data/Pilot Channels    | Yes         | No         | Yes       | Yes           |
| Modulation             | BPSK        | BPSK       | BOC/TMBOC | CBOC          |
| FEC Encoding           | LDPC        | None       | LDPC      | Convolutional |
| Code Length (chips)    | 2046/10,230 | 1023       | 10,230    | 4092          |
| Spreading Rate (Mcps)  | 1.023/5.115 | 1.023      | 1.023     | 1.023         |
| Code Period (ms)       | 2           | 1          | 10        | 4             |
| Data Symbol Rate (sps) | 500         | 50         | 100       | 250           |

### 3. Baseband Signal Generator

The LANS AFS baseband generator was developed using the open-source software GPS-SDR-SIM. It is a GPS signal simulation tool synthesizing the multiplexed GPS L1 C/A baseband signals as a user antenna receives them from multiple satellites. Originally released on GitHub in 2015 by one of the authors, Takuji Ebinuma, GPS-SDR-SIM has been widely adopted by the research community. The software has expanded to support other satellite navigation systems such as Galileo [13] and QZSS [14]. It has been utilized in a wide range of applications, from studies on GNSS signal security, including UAV spoofing [15,16], to the verification of emerging services such as Galileo Open Service Navigation Message Authentication (OSNMA) [17].

#### 3.1. Spreading Codes and Data

The existing GPS L1 C/A code generator can be easily adapted to produce the 2046-chip Gold code for the Data channel. Since the spreading code of the Pilot channel is identical to the GPS L1C primary code, the existing code generator can be used without any modification.

The AFS data frame structure also closely resembles that of GPS L1C, allowing for the reuse of several existing functions, including the CRC-24Q parity check and the BCH(52,9) encoder. The primary distinction lies in adopting a higher-performance LDPC code than that used in GPS L1C. While both systems employ a code rate of 1/2, the size of the parity-check matrix corresponding to a CED subframe in LANS AFS is  $5040 \times 6240$ , which is significantly larger compared to  $600 \times 1200$  in GPS L1C. This expansion of the base code is referred to as *lifting*, a technique utilized in the 5G New Radio (NR) standard for next-generation mobile networks [18]. Fortunately, the parity-check matrix is sparse, in which most of the elements are zero, allowing for efficient computation with existing sparse matrix libraries. In the present implementation, we leverage the sparse matrix operations provided by the LDPC library [19], also used in Pocket SDR.

Another notable feature is the use of *puncturing*. The lifted parity-check matrix generates a more extended code than the transmitted codeword. A subset of the encoded symbols is intentionally omitted (or punctured) from transmission to match the desired code length. During reception at the user receiver, these punctured symbols are treated as *erasures*, with log-likelihood ratio (LLR) values set to zero, indicating complete uncertainty. The FEC decoder then reconstructs the punctured symbols as part of the decoding process.

#### 3.2. Satellite Dynamics and Pseudorange

On each satellite, the I and Q components of the carrier are modulated with the Data and Pilot channel signals, respectively. However, these components are observed at the user receiver with a phase rotation, primarily caused by the Doppler effect resulting from the relative motion between the satellite and the user. Accordingly, the received signal  $s(t)$  can be expressed as follows:

$$s(t) = I(t) \cos(2\pi(f_c + \Delta f)t) - Q(t) \sin(2\pi(f_c + \Delta f)t) \quad (1)$$

where

$$I(t) = \sqrt{\frac{P}{2}} \cdot C_D(t - \tau) \cdot D(t - \tau) \quad (2)$$

$$Q(t) = \sqrt{\frac{P}{2}} \cdot C_P(t - \tau) \quad (3)$$

The I channel is modulated by the Data channel spreading code  $C_D(t)$  and the data symbol sequence  $D(t)$ . The layered spreading code  $C_P(t)$  modulates the Q channel.  $P$  is the signal transmission power,  $f_c$  is the center frequency, and  $\Delta f$  is the Doppler due to the satellite motion.  $\tau$  represents the time delay caused by the signal propagation.

By expanding and rearranging Equation (1) with respect to  $\Delta f$ , the following expression is obtained:

$$s(t) = I_B(t) \cos(2\pi f_c t) - Q_B(t) \sin(2\pi f_c t) \quad (4)$$

where,

$$I_B(t) = I(t) \cos(2\pi \Delta f t) - Q(t) \sin(2\pi \Delta f t) \quad (5)$$

$$Q_B(t) = I(t) \sin(2\pi \Delta f t) + Q(t) \cos(2\pi \Delta f t) \quad (6)$$

$I_B(t)$  and  $Q_B(t)$  are the I and Q components observed at the user receiver as the baseband signals.

During signal reception, the pseudorange is determined by measuring the time delay  $\tau$  between when the satellite transmitting the signal and when the user receiver received it. The signal transmission time  $t_F$  corresponding to the reception time  $t$  can be expressed as follows:

$$t_F = t - \tau \quad (7)$$

As shown in Equations (2) and (3), the signal transmission time  $t_F$  is derived from the code phase and navigation data symbol corresponding to the signal reception time. In the baseband signal generator,  $\tau$  is calculated based on the distance between the specified user location and the satellite position at the given reception time  $t$ .

In the baseband signal generator, the sampling frequency of each I/Q channel must be higher than the bandwidth of the spreading code. Consequently, computing the satellite position and corresponding time delay at every sampling point would impose an excessive computational load. To resolve this, the precise time delay is calculated only at a fixed time interval. The signal propagation time delay values between these time steps are then approximated by linear interpolation.

The signal propagation time delay  $\tau(t)$  at an arbitrary reception time  $t$  is calculated from the pseudorange  $\rho(t)$  as

$$\tau(t) = \frac{\rho(t)}{c} \quad (8)$$

where  $c$  is the speed of light. The pseudorange itself is given by

$$\rho(t) = r(t) - c \cdot dT \quad (9)$$

where  $r(t)$  is the geometric range between the receiver and the satellite at time  $t$ , and  $dT$  is the satellite clock offset.

To reduce computational load, the pseudorange at each sampling point  $t_i$  is linearly interpolated as

$$\rho(t_{i+1}) = \rho(t_i) + \Delta\rho \cdot \delta t \quad (10)$$

where  $\delta t$  is the sampling interval and  $\Delta\rho$  is the averaged pseudorange rate during the fixed interpolation interval  $\Delta T$ . The averaged pseudorange rate is computed as

$$\Delta\rho = \frac{\rho(t_i + \Delta T) - \rho(t_i)}{\Delta T} \quad (11)$$

This approximation also facilitates the computation of the Doppler frequency  $\Delta f$  used in Equations (5) and (6) as

$$\Delta f = -\frac{\Delta \rho}{\lambda} \quad (12)$$

where  $\lambda$  is the wavelength of the carrier signal.

In the current implementation, the interpolation interval  $\Delta T$  is set to 100 ms. This time interval provides a good balance by significantly reducing computational costs while maintaining sufficient accuracy to approximate the nonlinear variations in time delay caused by the motion of the receiver and the satellites.

The code phase and data symbol are then determined from the computed transmission time as follows:

$$t_F = WN \cdot SECWEEK + ITOW \cdot BI_d + TOI \cdot F_d + \Delta t \quad (13)$$

where

- $WN$  is an integer counter indicating the sequential week number from the start epoch of the LunaNet Reference Time (LRT).
- $SECWEEK$  is the number of seconds in one week (i.e., 604,800 s).
- $ITOW$  (interval time of week) is the number of block intervals that have occurred since the start of the current week.
- $BI_d$  is the block interval duration, equal to 1200 s.
- $TOI$  (time of interval) is the number of frames from the beginning of the current block interval.
- $F_d$  is the frame duration, corresponding to 6000 data symbols, or 12 s.

Together,  $ITOW$  and  $TOI$  determine the integer number of seconds that have elapsed since the beginning of the week.  $WN$ ,  $ITOW$ , and  $TOI$  are encoded in the Time of Transmission (ToT) message within the navigation data and can be retrieved by a receiver once it successfully tracks the spreading code.

$\Delta t$  represents the fractional seconds within the current frame. It is computed by adding the duration corresponding to the integer count of data symbols from the start of the frame and the code phase obtained through correlation. Specifically, the data symbol index is multiplied by the symbol duration corresponding to the code period (2 ms), and the code phase is scaled by the code period.

As of the latest AFS specification, the LRT has not yet been formally defined. Therefore, the baseband signal generator implementation uses GPS time as an alternative. Similarly, the lunar coordinate system and satellite orbit models have not been defined. As a result, a simplified model of the Moon is used, assuming it is a perfect sphere. Furthermore, the Moon is supposed to be non-rotating, so the Moon-fixed coordinate system coincides with an inertial reference frame. Based on these assumptions, satellite positions are computed using Keplerian orbits. Thanks to the flexibility of SDR-based systems, once these definitions are finalized, they can be easily updated.

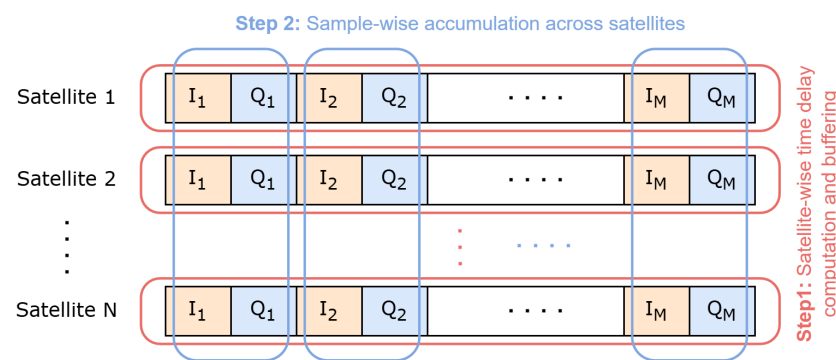
### 3.3. Parallelization with OpenMP

Although linear interpolation of the time delay helps reduce the computational load, the spreading code bandwidth of the AFS Pilot channel exceeds 10 MHz. It requires a much higher sampling frequency than GPS L1 C/A, which has a bandwidth of approximately 2 MHz. To address this, the LANS AFS baseband signal generator employs multi-core processing with OpenMP.

OpenMP is an API that supports multi-platform shared-memory parallel programming in C, C++, and Fortran [20]. It provides a flexible interface for developing parallel applications on multi-core processors. Since parallelization is achieved automatically

through compiler directives, OpenMP preserves code readability, making it easier to maintain and develop parallel programs.

Parallelization using OpenMP is applied in two separate stages. First, since the time delay calculation of each satellite is independent, this process is parallelized across satellites. The resulting I/Q sample values, derived from Equations (5) and (6) at each sampling time  $t$  with the corresponding time delay  $\tau$  are then stored in buffers at fixed intervals for each satellite. In the second stage, the buffered I/Q samples are summed across all satellites at each sampling point to generate the final composite signal. Because the computations at each sampling point are also independent, this accumulation process is further parallelized along the timeline of sampling. Figure 1 provides an overview of the two-stage parallelization accelerating baseband signal generation.



**Figure 1.** Two-stage parallelization of baseband signal generation. In the first stage (red boxes), the time delays and I/Q samples of each satellite are computed and buffered independently. In the second stage (blue boxes), the buffered I/Q samples are accumulated across all satellites at each sampling point to produce the composite signal.

Table 2 summarizes the performance improvement in baseband signal generation achieved through parallelization with OpenMP. The processing time increases linearly with the data length of the generated I/Q samples. Even without OpenMP, the generator can produce data in less time than the intended duration, which is necessary for running the LANS AFS simulator in real time. Furthermore, by adding only two compiler directives to the nominal source code, OpenMP parallelization achieves a nearly sixfold speedup across all data lengths. This results in more than ten times faster than real-time performance, providing ample margin for efficient baseband signal generation.

**Table 2.** Processing time comparison with and without OpenMP parallelization.

| Data Length (s) | Processing Time (s) |        |              |
|-----------------|---------------------|--------|--------------|
|                 | Serial              | OpenMP | Improvement  |
| 30              | 14.4                | 2.5    | $\times 5.8$ |
| 60              | 29.0                | 5.0    | $\times 5.8$ |
| 90              | 43.8                | 7.6    | $\times 5.8$ |

The performance evaluation of the OpenMP-based baseband signal generator was conducted on a PC equipped with an Intel Core i9-10900X processor (10 cores, 20 threads, base frequency 3.7 GHz) and 64 GB of RAM. The program was built and executed in the MSYS2/MinGW64 environment on Windows 11.

## 4. LANS AFS Receiver

In our work, Pocket SDR served as the foundation for developing a functional LANS AFS receiver. Pocket SDR is an open-source GNSS software-defined receiver framework developed and maintained by Tomoji Takasu, the creator of the well-known RTKLIB [21]. Designed for portability, accessibility, and educational use, Pocket SDR provides a complete, modular pipeline for GNSS signal acquisition, tracking, and navigation processing. Unlike traditional SDR GNSS receivers, which typically require high-performance desktop PCs, PocketSDR has been successfully demonstrated on embedded platforms such as Raspberry Pi 5, which is particularly well-suited for SWaP-constrained systems.

### 4.1. Adaptation for LANS AFS Reception

Similar to the baseband signal generator, the spreading codes for both the Data and Pilot channels of LANS AFS can be generated using the existing generators in Pocket SDR, either without modification or with minimal changes. Furthermore, Pocket SDR is designed with high extensibility to support new signal types. Once the spreading code generators are implemented, signal acquisition and tracking can be performed without additional modifications by specifying parameters such as code length, chip rate, and center frequency.

Implementing the navigation data decoder for LANS AFS is also primarily based on the existing architecture of the CNAV-2 decoder, which is used for the GPS L1C signal. Similar to the encoder implementation discussed in Section 3.1, the primary difference between these decoders lies in the parity-check matrix used by the LDPC decoder. It utilizes the same  $5040 \times 6240$  parity-check matrix as the LDPC encoder used during signal generation to reconstruct the original navigation data symbols. The decoder operates by iteratively processing the received symbols to detect and correct transmission errors. This error correction process significantly enhances the robustness and reliability of data transmission, especially under low signal-to-noise conditions. Another important consideration is that, besides the symbols in the navigation data frame, the decoder must also restore the *punctured* symbols that were omitted during transmission. These are treated as *erasure* symbols, with zero LLR values indicating an equal probability of being 0 or 1. These symbols are corrected through the powerful FEC capabilities of the LDPC decoder.

Once signal tracking is established via a Phase-Locked Loop (PLL), navigation data symbols are extracted sequentially. In LANS AFS, the symbol duration is equal to the length of one spreading code of the Data channel. This eliminates the need for a bit synchronization process typically required in GPS signal processing. Unlike the CNAV-2 format used in the GPS L1C signal, each navigation frame of LANS AFS begins with a synchronization pattern (SP) consisting of 68 unencoded symbols. This unencoded preamble allows direct use of the SP for frame synchronization without requiring prior decoding.

The current implementation performs frame synchronization and navigation data extraction through the following sequence:

1. **Buffering:** Symbols obtained from the PLL are stored in a FIFO buffer sized to accommodate one full navigation frame plus the SP (a total of  $6000 + 68$  symbols).
2. **Synchronization Check:** If the initial 68 symbols in the FIFO buffer match the pre-defined SP, the implementation checks whether the SP also appears at the expected position one frame length later. If both instances match, frame synchronization is declared successful.
3. **TOI Extraction:** Upon successful synchronization, Subframe 1 (SB1), encoded with a BCH(52,9) code, is decoded in an attempt to extract the *TOI*. If decoding fails or the *TOI* cannot be determined, the process reverts to Step 1.
4. **CED Decoding:** If the *TOI* is successfully identified, Subframe 2 (SB2), which contains the Clock and Ephemeris Data (CED), is decoded using an LDPC decoder. If

LDPC decoding fails, the process restarts from the beginning at Step 1. Subframes 3 and 4, currently undefined in the LSIS-AFS specification, are not processed in this implementation.

5. CRC Verification: A cyclic redundancy check (CRC) is performed on the decoded SB2. If the CRC check fails, the system returns to the initial step. Otherwise, the CED is accepted and made available for positioning computation.

The *TOI* in Step 3 is determined by comparing the received SB1 symbols with a precomputed lookup table containing all possible encoded *TOI* values ranging from 0 to 99. This approach efficiently identifies the *TOI* without the BCH(52,9) decoder.

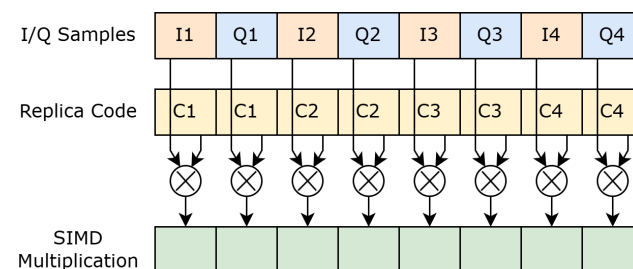
By decoding SB2, the *WN* and the *ITOW*, which are necessary for determining the signal transmission time as shown in Equation (13), can be obtained in addition to the *TOI* derived from SB1. Together with the data symbol count from the start of the synchronized frame and the code phase delay within one spreading code period obtained from the correlator in the Delay-Locked Loop (DLL), the pseudorange can be calculated. Furthermore, using the CED decoded from SB2, the satellite position and clock error can be computed, thereby enabling the derivation of the navigation solution.

Pocket SDR utilizes functions from RTKLIB to compute the navigation solutions. However, RTKLIB is designed exclusively for terrestrial positioning. Due to its large and complex code structure, adapting the entire RTKLIB for lunar positioning without compromising existing GNSS capabilities is a challenging task. Therefore, for demonstration purposes, a dedicated lunar positioning function was implemented without RTKLIB to compute positions on the same spherical Moon defined in the baseband signal generator.

#### 4.2. SIMD Optimization

GNSS frontends commonly employ 2-bit analog-to-digital converters (ADCs) to digitize received RF signals into baseband I/Q samples. A standard 2-bit ADC encodes signals into the four quantized values of  $-3$ ,  $-1$ ,  $+1$ , and  $+3$ .

Pocket SDR leverages this low-resolution quantization format to accelerate PRN code correlation and carrier mixing operations using SIMD (Single Instruction, Multiple Data) instructions. SIMD is a CPU instruction set that allows the same arithmetic operation to be applied simultaneously to multiple pieces of data in a single instruction cycle. It is well-suited for processing extensive data with simple, uniform operations. For example, a part of the GNSS correlator can be implemented using a simple multiplication operation between the digitized I/Q samples and a locally generated PRN code. This operation can be efficiently parallelized using SIMD. As shown in Figure 2, four sets of I/Q samples are packed into a single SIMD register while corresponding replica PRN code values are loaded into another. A single SIMD instruction then performs parallel multiplication across all data pairs.



**Figure 2.** SIMD-based acceleration of GNSS signal correlation. Four sets of digitized I/Q samples and replica PRN code values are packed into SIMD registers and processed in parallel using a single SIMD instruction.

This concept of SIMD-based parallel signal processing in GNSS software receivers was first introduced by Ledvina et al. in 2004 [22]. It inspired subsequent SDR implementations, including RadioLion from the University of Texas at Austin, another Raspberry Pi-based software-defined GNSS receiver [23,24]. Thanks to its SIMD-optimized signal processing architecture, Pocket SDR runs efficiently even on low-power platforms. On a quad-core ARM-based processor such as the Raspberry Pi 5, Pocket SDR can simultaneously track up to 12 Data and 12 Pilot channels at a 12 MHz sampling rate. It allows real-time, all-in-view LANS AFS signal reception without the need for high-performance desktop PCs.

The SIMD optimizations applied to functions used in Pocket SDR are summarized in Table 3. These functions are implemented in the file `src/sdr_func.c`. Depending on the CPU architecture, there are two SIMD instruction sets: AVX2 and NEON. AVX2 is an Intel/AMD SIMD instruction set extension for x86 CPUs that operates on 256-bit vector registers, enabling high-performance parallel processing. NEON is an SIMD instruction set for ARM processors that provides efficient parallel processing using 128-bit vector registers.

**Table 3.** SIMD-optimized functions in Pocket SDR and their CPU architecture support.

| Function                 | Description                                     | AVX2 | NEON |
|--------------------------|-------------------------------------------------|------|------|
| <code>sdr_cpx_mul</code> | Multiplication of two complex arrays            | Yes  | No   |
| <code>mix_carr</code>    | Carrier mixing for carrier wipe-off             | Yes  | No   |
| <code>sum_s16</code>     | Summation of multiple <code>int16</code> fields | Yes  | Yes  |
| <code>dot_IQ_code</code> | Inner product of I/Q samples and local code     | Yes  | Yes  |

As shown in Table 3, some functions have not yet been SIMD-optimized using NEON. Although applying NEON SIMD instructions to these functions is technically possible, the expected performance improvement has not been achieved on the Raspberry Pi 5. One possible reason is the slower memory access overhead compared to a typical desktop PC. While the current implementation is already capable of real-time positioning on the Raspberry Pi 5, further SIMD optimization using NEON remains a future challenge for improving performance.

#### 4.3. Frontend Devices

The only hardware required before software-based signal processing is a frontend device, which captures incoming radio frequency signals and converts them into a digital signal for further processing in an SDR. While typical GNSS signals are broadcast in the L-band, the LANS AFS is transmitted in the S-band with a center frequency of 2492.028 MHz. Therefore, a dedicated S-band frontend device is required for Pocket SDR to receive the LANS AFS.

One candidate for such a device is the NTLab NT1066. It is a four-channel RF frontend IC that supports all GNSS signals, with one channel specifically designed to operate in the S-band. Since it provides 2-bit ADC outputs, it can be easily replaced with the general GNSS frontend ICs. The S-band performance is currently being evaluated using NTLab's development kit.

Another option is the AD9364, a programmable RF transceiver IC from Analog Devices. The AD9364 operates over a wide frequency range from 70 MHz to 6 GHz and supports channel bandwidths of up to 56 MHz. It has been adopted by many SDR products, such as Ettus Research's USRP B200 series, and readily accommodates traditional GNSS signals in the L-band and LANS AFS in the S-band. However, these transceiver ICs provide baseband signals quantized with a 12-bit ADC, while Pocket SDR requires 2-bit I/Q samples for efficient correlation processing using SIMD. Therefore, to use these general-purpose RF

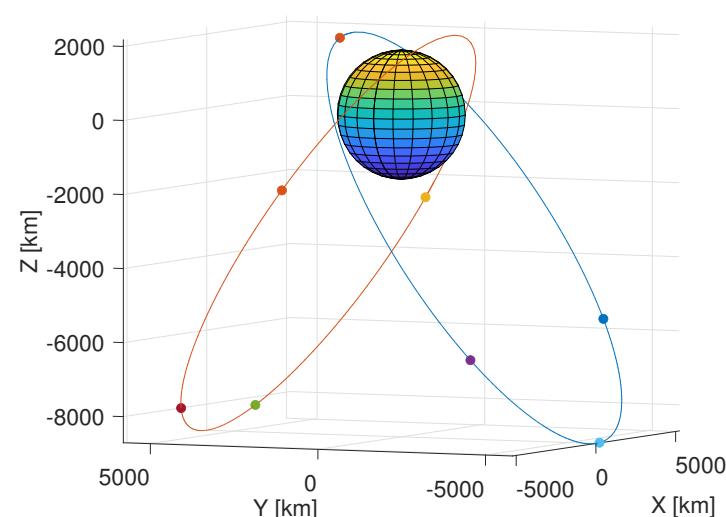
transceiver ICs with Pocket SDR, a digital automatic gain controller (AGC) is needed to convert the 12-bit baseband signals into 2-bit ADC values.

These S-band frontend devices are still under development at the time of writing, and their operation with Pocket SDR has not yet been verified. Therefore, in the present demonstration, the LANS AFS baseband signal was transmitted at 1575.42 MHz, the same frequency as the GPS L1 C/A signal. This L-band signal can naturally be received by Pocket SDR using an existing frontend device, FE 2CH, which incorporates the MAX2771 chips from Analog Devices. The circuit schematic and PCB design of FE 2CH are openly available in the Pocket SDR repository on GitHub.

## 5. Validation Tests

Validation tests were conducted offline and in real time to evaluate the functionality and performance of the proposed LANS AFS simulator and receiver system. The offline tests were used primarily for algorithm verification and debugging, while the real-time tests demonstrated the operational capability of the complete system using actual hardware.

The simulated satellite orbit and the user receiver position were identical in the offline and real-time validation tests. The user receiver position was set at the Shackleton Crater in the lunar south polar region [25]. The floor of this crater contains areas of permanent shadow that never receive sunlight and are extremely cold, where water is believed to exist in the form of ice. As such, it is considered one of the most promising candidate sites for future lunar exploration. The satellite orbits were modeled as Elliptical Lunar Frozen Orbits (ELFOs) based on the proposed Lunar Navigation Satellite System (LNSS) architecture developed by JAXA [26,27]. The simulated constellation consists of eight satellites evenly distributed across two orbital planes, with four satellites per plane. Each orbital plane has a semi-major axis of 6540 km and an eccentricity of 0.6. This high-eccentricity configuration ensures long visibility over the lunar south pole, particularly target regions such as Shackleton Crater. Figure 3 illustrates the configuration of JAXA's LNSS satellite constellation used in our simulations.



**Figure 3.** LNSS satellite constellation with eight satellites in two ELFO planes, optimized for coverage over the lunar south pole.

### 5.1. Offline Test Results

To support efficient algorithm development and debugging, offline validation of the LANS AFS simulator and receiver was conducted using a high-performance desktop PC. In this test, I/Q samples generated by the baseband signal generator were saved to a file and fed directly into Pocket SDR. Because the role of the RF frontend is limited to digitizing

the analog RF signal into I/Q samples, the signal processing pipeline within Pocket SDR performs identically whether the input is received via live hardware or from a file. The offline test enables the comprehensive testing of Pocket SDR's software components, including acquisition, tracking, synchronization, and navigation message decoding without requiring real-time signal transmission or frontend hardware.

Since most commercially available general-purpose SDR devices utilize a 16-bit complex format for their 12-bit ADC input and output interfaces, the default I/Q sample format of the LANS AFS baseband generator is set to the same 16-bit complex format, in which the I and Q components are interleaved as signed 16-bit integers. In contrast, Pocket SDR requires 2-bit I/Q samples as input. Therefore, as described in Sections 4.2 and 4.3, the baseband signal generated for the offline test must be adequately converted into 2-bit ADC values. Moreover, the I/Q samples generated by the baseband signal generator contain only the ideal signal component and do not include any noise. Consequently, in offline validation tests where I/Q data is exchanged via files, adding noise to the I/Q samples is necessary according to a desired signal-to-noise ratio (SNR). The SNR is a critical parameter that directly affects the accuracy of pseudorange measurements and the positioning solutions.

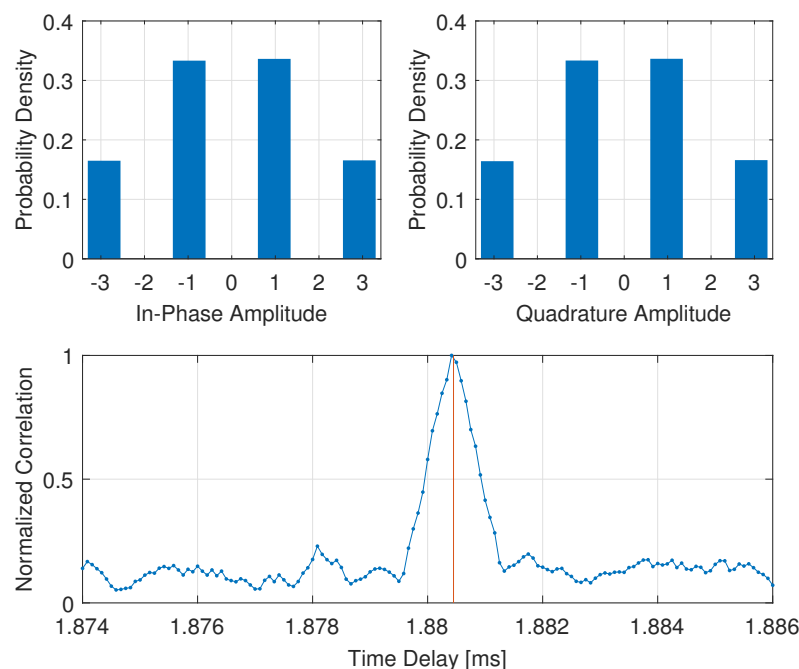
Since the noise values are computed at every sampling, the noise generator algorithm must perform efficiently. For that reason, the Central Limit Theorem (CLT) method is selected to generate a noise-like signal. It approximates a Gaussian (normal) distribution by summing multiple uniformly distributed random values. Although the CLT is less accurate than more precise methods, such as the Box–Muller transform, it offers significantly faster performance. The uniform random numbers are generated using a 32-bit linear feedback shift register (LFSR), and six samples are accumulated for each output to approximate a Gaussian noise value.

After Gaussian noise is added to the simulated baseband signal, the resulting waveform is quantized into 2-bit ADC values. In typical LANS AFS reception scenarios, the signal power is much lower than the thermal noise, meaning the received signal appears noise-like and follows a Gaussian distribution. For 2-bit quantization, threshold levels are set based on the standard deviation of the received signal [28]. With this threshold, the effective signal power loss is limited to approximately 0.55 dB, resulting in negligible degradation in navigation performance. Specifically, the 2-bit ADC maps input signals to  $\pm 1$  for values within one standard deviation from the mean and  $\pm 3$  for values beyond that range. As a result, approximately 70% of the I/Q samples fall into the  $\pm 1$  range, while the remaining 30% are assigned to  $\pm 3$ .

Figure 4 presents an example of the offline signal acquisition results using simulated 2-bit I/Q samples of the LANS AFS signal. The top panels show histograms of the ADC values for the I and Q components. Both distributions exhibit the expected Gaussian profile, indicating that the 2-bit ADC effectively captures the noise-like characteristics of the received signal, which affects the in-phase and quadrature components equally. The bottom panel displays the result of the code correlation process for one of the Data channel signals. A distinct triangular correlation peak indicates a successful signal acquisition. The code phase at the peak can be used to compute the fractional time delay within one spreading code period, as shown in Equation (13).

Figure 5 shows a screenshot of the offline operation of the `pocket_trk` application of Pocket SDR. The first row displays the current time, navigation solution, and the number of satellites used for positioning. From the second row onward, each line represents a tracking channel with the following fields: CH (channel number), RF (signal input port), SAT (satellite number), and SIG (signal type). Here, AFSD refers to the Data channel and AFSP to the Pilot channel. C/N0 denotes the carrier-to-noise density ratio, COFF indicates the time

delay estimated from the spreading code correlation, and DOP shows the Doppler frequency of the received signal.



**Figure 4.** Acquisition results for simulated 2-bit LANS AFS I/Q samples. (Top) Histograms of I and Q values showing Gaussian-like distributions. (Bottom) Data channel correlation output with a clear triangular peak indicating code phase delay.

```
Galileo@GPSLAB MINGW64 ~/PocketSDR/PocketSDR_AFS/app/pocket_trk
$ ./pocket_trk.sh ~/lans-afs-sim/test.bin
FFTW wisdom import error ../python/fftw_wisdom.txt
2023-08-30 16:45:28.002 -89.6599939 129.1993581 100.84 7/ 7 FIX
CH RF SAT SIG PRN LOCK(s) C/N0 (dB-Hz) COFF (ms) DOP (Hz)
1 1 L02 AFSD 2 40.80 41.3 ||||| 0.6375246 -2377.0
2 1 L03 AFSD 3 40.61 39.5 ||||| 0.2709383 5.5
3 1 L04 AFSD 4 40.44 41.4 ||||| 0.5863138 2386.2
4 1 L05 AFSD 5 40.08 43.5 ||||| 1.9294837 -3009.3
5 1 L06 AFSD 6 39.93 40.0 ||||| 0.8573587 -1173.4
6 1 L07 AFSD 7 39.74 40.1 ||||| 0.8061874 1195.1
7 1 L08 AFSD 8 39.57 44.1 ||||| 1.7898190 3004.8
8 1 L02 AFSP 2 39.40 41.6 ||||| 0.6373477 -2376.8
9 1 L03 AFSP 3 39.22 39.6 ||||| 0.2709397 5.0
10 1 L04 AFSP 4 39.04 41.3 ||||| 0.5864891 2386.2
11 1 L05 AFSP 5 38.84 44.0 ||||| 1.9294842 -3008.9
12 1 L06 AFSP 6 38.66 40.2 ||||| 0.8573604 -1173.5
13 1 L07 AFSP 7 38.49 40.1 ||||| 0.8061939 1195.1
14 1 L08 AFSP 8 38.32 44.2 ||||| 1.7898176 3004.4
```

**Figure 5.** Screenshot of an offline test, showing signal tracking and navigation results using simulated 2-bit LANS AFS I/Q data.

Seven of the eight simulated satellites are visible in this offline test. In the current implementation, only the pseudoranges obtained from the Data channels are used for the navigation solution. In contrast, the Pilot channels are tracked but are not used in the computation. Leveraging the Pilot channel to enhance navigation performance is considered future work. The C/N0 values reflect appropriate signal strengths corresponding to the propagation distances. The navigation data is successfully decoded, and the computed position matches the user receiver location at the Shackleton Crater specified in the baseband signal generator. These offline test results confirm that Pocket SDR successfully supports LANS AFS.

## 5.2. Real-Time Test Results

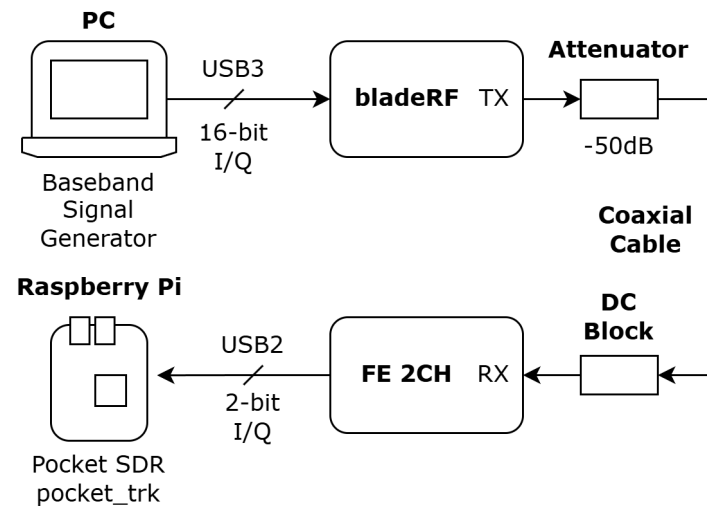
After confirming the correct operation in the offline environment, real-time validation was conducted to evaluate the complete system performance using actual hardware. The `pocket_trk` application, extended to support LANS AFS and verified in the offline test, was successfully built and deployed on a Raspberry Pi 5. In this configuration, I/Q samples were streamed directly from a connected frontend device, replacing the file-based input used in offline testing. This setup enables real-time, continuous signal tracking and position computation, demonstrating the feasibility of a fully functional LANS AFS receiver using a compact hardware platform.

As mentioned in Section 4.3, S-band-capable hardware is still under development at the time of writing. Therefore, real-time validation was conducted using the L-band frontend device FE 2CH, which has already been verified to work with Pocket SDR. The baseband signal was generated with a center frequency of 1575.42 MHz, identical to that of GPS L1 C/A. A bladeRF device from Nuand was used as the frontend on the transmitter side for signal transmission. Although the validation was performed in the L-band, the signal processing within the receiver does not fundamentally differ from that of the S-band. The only difference lies in the Doppler shift, which depends on the carrier frequency. This effect is handled flexibly within the software and does not require hardware modifications. Therefore, once an S-band-capable frontend becomes available, it can be seamlessly substituted without changes to the receiver implementation. This minimal hardware dependency is a key advantage of SDR-based architecture, enabling early validation of signal processing functions even in the absence of dedicated S-band hardware.

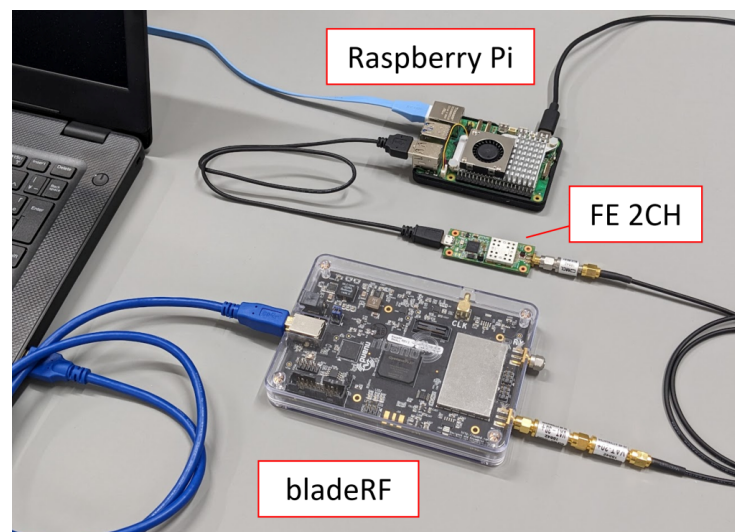
Figure 6 presents a block diagram of the real-time test platform. Baseband I/Q sample data generated by the signal generator is transmitted to the bladeRF device via USB 3.0 using the `bladeRF-cli` tool provided by Nuand [29]. This command-line utility enables configuration and control of the bladeRF, including the transmission of baseband waveforms. To ensure that the transmitted signal power matches the expected LANS AFS reception level, a combination of the variable TX amplifier on the bladeRF and external RF attenuators is employed. The signal is then fed through a coaxial cable to the FE 2CH receiver frontend, where it is down-converted to the baseband and quantized into 2-bit I/Q samples. These samples are captured by a Raspberry Pi 5 over USB 2.0, and the `pocket_trk` application running on the Raspberry Pi 5 performs real-time tracking and navigation based on the received LANS AFS signals.

Figure 7 shows a photo of the actual real-time setup used in this experiment. This physical setup corresponds to the functional block diagram shown in Figure 6 and demonstrates real-time operation on embedded hardware. In this configuration, the Raspberry Pi 5 is additionally connected to a local area network (LAN) for monitoring the real-time positioning results.

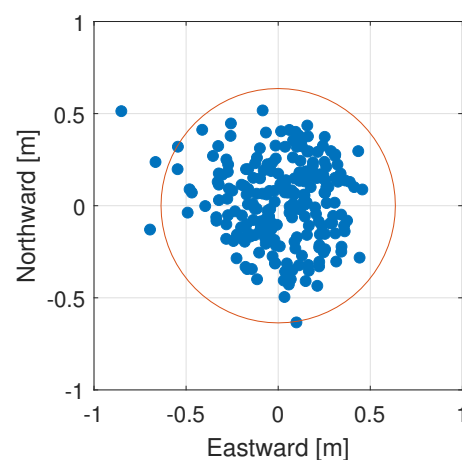
Figure 8 shows the real-time positioning results using the `pocket_trk` application running directly on Raspberry Pi 5, without relying on external processing. In this test, the positioning computation was performed every second. The simulation scenario was identical to that of the offline tests, featuring seven satellites visible and utilized in the positioning solution. Pocket SDR employs an FFT-based signal acquisition engine, enabling efficient and rapid detection of satellite signals. All seven satellite signals were acquired within 7 s from a cold start in the real-time test scenario. Thanks to the sufficient number of tracked satellites and favorable SNR, the horizontal positioning accuracy achieved was 0.64 m (2drms), indicating good navigation performance.



**Figure 6.** Functional block diagram of the real-time LANS AFS receiver test setup using Raspberry Pi 5 for SDR-based signal processing.

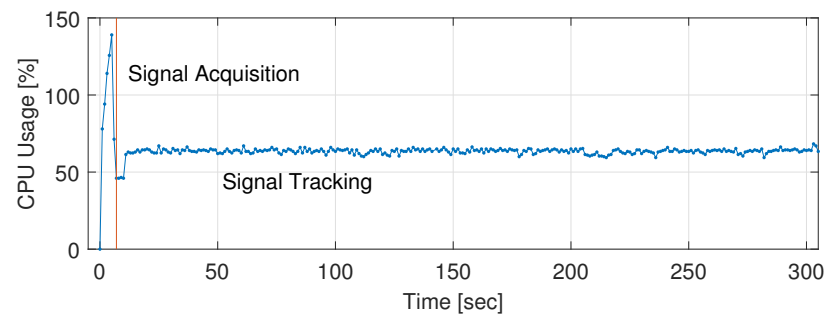


**Figure 7.** Physical test setup for the real-time LANS AFS receiver test setup using the proposed SDR platform.



**Figure 8.** Horizontal position errors (blue dots) relative to the simulated reference location. The red circle indicates the 2 drms boundary, representing the region within which approximately 95% of the position estimates are contained.

Although only the Data channel signals were used for positioning, the sampling frequency of the FE 2CH frontend was set to 12 MHz to enable tracking of the wider-bandwidth Pilot channel signals as well. Under this configuration, the CPU usage of the `pocket_trk` application on the Raspberry Pi 5, as monitored using the `top` command, remained around 64% of a single core, as shown in Figure 9. This excludes the brief initial signal acquisition phase, during which CPU usage temporarily increases. Since the Raspberry Pi 5 has a quad-core ARM processor with a total of 400% acceptable CPU usage, there is substantial processing headroom. This indicates that the receiver can accommodate a larger constellation without exceeding its processing limits, enabling the future integration of advanced functions, such as Pilot-assisted tracking and multipath mitigation.



**Figure 9.** CPU usage of the LANS AFS receiver on the Raspberry Pi 5. The load remains around 64% of a single core during tracking, excluding the initial acquisition phase indicated by the red line.

The real-time test successfully demonstrated the feasibility of a fully functional LANS AFS receiver using a compact, low-power hardware platform. Although Raspberry Pi hardware is not formally space-qualified, various Raspberry Pi models have been successfully deployed in space missions, demonstrating a degree of space-readiness in practice [30]. Furthermore, the SDR-based LANS AFS receiver is implemented entirely in portable C code and is compatible with standard GCC toolchains. Thus, the software can be readily ported to other platforms, including space-qualified hardware systems that support a Linux operating environment.

## 6. Conclusions

In this work, we present the development and validation of a comprehensive SDR-based solution for simulating and receiving the newly defined LANS AFS signal, a key component of the LunaNet lunar navigation system. By leveraging and extending open-source tools such as GPS-SDR-SIM and Pocket SDR, we achieved rapid prototyping and real-time performance on compact embedded platforms like the Raspberry Pi 5. Our LANS AFS receiver supports signal reception and positioning without reliance on high-performance desktop hardware, making it highly suitable for SWaP-constrained lunar applications.

Extensive offline and real-time validation confirmed that the LANS AFS simulator and receiver functioned correctly, achieving meter-level positioning accuracy in a lunar scenario. The modular design and open-source components enable flexible adaptation for future updates of the CED format and the lunar reference time and coordinate systems.

While our entirely software-based SDR receiver offers excellent flexibility for LANS AFS receiver development and testing, it poses challenges in precise timing control. Unlike hardware implementations using FPGAs or ASICs, software systems encounter difficulties in generating tightly synchronized timing signals, such as pulse-per-second (PPS) outputs. As a result, the latency from signal reception to pseudorange computation and delivery of the navigation solution cannot be strictly bounded. Although this latency is acceptable for moderate-speed platforms such as lunar rovers, it may prove inadequate for high-dynamics

scenarios like the descent phase of a lunar lander. To support time-critical applications, offloading certain SDR processing functions to hardware accelerators may be necessary. For example, recent work has demonstrated the generation of PPS using SDR for GNSS signal reception by integrating FPGA-based timing logic [31]. However, these approaches require additional hardware and increase system dependencies on a specific platform.

Future work will focus on several areas to enhance system performance and applicability. Additional SIMD optimization using the NEON instruction set will improve processing efficiency on ARM-based embedded platforms. Integration with dedicated S-band RF frontend hardware will complete the signal chain and enable nominal LANS AFS reception. Further developments can expand Pilot-assisted tracking to improve support for low-SNR operations and introduce lunar-specific navigation solvers with multipath mitigation capability.

This work demonstrates that SDR-based architectures provide a flexible and practical platform for future lunar navigation systems. We aim to encourage community collaboration and accelerate innovation in lunar PNT technologies by releasing the simulator and receiver as open-source software.

**Author Contributions:** Conceptualization, R.S. and T.E.; methodology, R.S. and T.E.; software, R.S. and T.E.; validation, R.S. and T.E.; formal analysis, R.S. and T.E.; investigation, R.S. and T.E.; resources, T.E.; data curation, R.S. and T.E.; writing—original draft preparation, R.S.; writing—review and editing, T.E.; visualization, R.S. and T.E.; supervision, T.E.; project administration, T.E.; funding acquisition, T.E. All authors have read and agreed to the published version of the manuscript.

**Funding:** This research was supported by JAXA SSF Program Japan Grant Number JPJXSSF24ME13001.

**Institutional Review Board Statement:** Not applicable.

**Informed Consent Statement:** Not applicable.

**Data Availability Statement:** The software implementations developed in this study are freely available as open-source projects. The LANS AFS signal simulator is available at <https://github.com/osqzss/LANS-AFS-SIM> (accessed on 27 May 2025), and the modified Pocket SDR receiver supporting LANS AFS is available at <https://github.com/osqzss/PocketSDR-AFS> (accessed on 27 May 2025).

**Acknowledgments:** The authors would like to express their sincere gratitude to Tomoji Takasu for his outstanding contribution to the GNSS research community through the development and open-source release of Pocket SDR. This work would not have been possible without the extensibility, performance, and accessibility provided by Pocket SDR, which served as the foundation for our LANS AFS receiver development.

**Conflicts of Interest:** The authors declare no conflicts of interest.

## Abbreviations

The following abbreviations are used in this manuscript:

|      |                                   |
|------|-----------------------------------|
| ADC  | Analog-to-Digital Converter       |
| AFS  | Augmented Forward Signal          |
| AGC  | Automatic Gain Control            |
| API  | Application Programming Interface |
| BPSK | Binary Phase Shift Keying         |
| CBOC | Composite Binary Offset Carrier   |
| CED  | Clock and Ephemeris Data          |
| CLT  | Central Limit Theorem             |
| CNAV | Civil Navigation Message          |
| CPU  | Central Processing Unit           |

|      |                                               |
|------|-----------------------------------------------|
| CRC  | Cyclic Redundancy Check                       |
| DLL  | Delay-Locked Loop                             |
| ELFO | Elliptical Lunar Frozen Orbits                |
| ESA  | European Space Agency                         |
| FEC  | forward error correction                      |
| GNSS | Global Navigation Satellite System            |
| GPS  | Global Navigation System                      |
| ITOW | Interval Time of Week                         |
| JAXA | Japan Aerospace Exploration Agency            |
| LANS | Lunar Augmented Navigation Service            |
| LDPC | Low Density Parity Check                      |
| LFSR | Linear Feedback Shift Register                |
| LLR  | Log-Likelihood Ratio                          |
| LNIS | LunaNet Interoperability Specification        |
| LNSP | LunaNet Service Provider                      |
| LNSS | Lunar Navigation Satellite System             |
| LRT  | LunaNet Reference Time                        |
| NASA | National Aeronautics and Space Administration |
| PLL  | Phase-Locked Loop                             |
| PNT  | Position, Navigation, and Timing              |
| PRN  | Pseudorandom Noise                            |
| SDR  | Software Defined Radio                        |
| SIMD | Single Instruction, Multiple Data             |
| SNR  | Signal-to-Noise Ratio                         |
| SP   | Synchronization Pattern                       |
| SWaP | Size, Weight, and Power                       |
| TOI  | Time of Interval                              |
| ToT  | Time of Transmission                          |
| WN   | Week Number                                   |

## References

1. Israel, D.J.; Mauldin, K.D.; Roberts, C.J.; Mitchell, J.W.; Pulkkinen, A.A.; Cooper, L.V.D.; Johnson, N.A.; Christe, S.D.; Gramling, C.J. LunaNet: A Flexible and Extensible Lunar Exploration Communications and Navigation Infrastructure. In Proceedings of the 2020 IEEE Aerospace Conference, Big Sky, MT, USA, 7–14 March 2020; p. 1. [\[CrossRef\]](#)
2. NASA. LunaNet Signal-in-Space Recommended Standard—Augmented Forward Signal, Value A, Version 1. 20 January 2025. Available online: <https://www.nasa.gov/wp-content/uploads/2025/02/lunanet-signal-in-space-recommended-standard-augmented-forward-signal-vol-a.pdf> (accessed on 12 May 2025).
3. NASA. LunaNet Interoperability Specification, Version 5. 29 January 2025. Available online: <https://www.nasa.gov/wp-content/uploads/2025/02/lunanet-interoperability-specification-v5-baseline.pdf> (accessed on 12 May 2025).
4. De Oliveira Salgueiro, F.; Melman, F.T.; Swinden, R.; Audet, Y.; Giordano, P.; Ventura-Traveset, J. A Novel Navigation Message for Future LCNS Satellites. *Eng. Proc.* **2025**, *88*, 52. [\[CrossRef\]](#)
5. Bury, G.; Zajdel, R.; Sośnica, K. Design of the broadcast ephemerides for the Lunar Communication and Navigation Services system. *Prog. Earth Planet Sci.* **2025**, *12*, 20. [\[CrossRef\]](#)
6. Pany, T.; Akos, D.; Arribas, J.; Bhuiyan, M.Z.H.; Closas, P.; Dovis, F.; Fernandez-Hernandez, I.; Fernández-Prades, C.; Gunawardena, S.; Humphreys, T.; et al. GNSS Software-Defined Radio: History, Current Developments, and Standardization Efforts. *Navigation* **2024**, *71*, 1. [\[CrossRef\]](#)
7. Pocket SDR. Available online: <https://github.com/tomokitaku/PocketSDR> (accessed on 12 May 2025).
8. GPS-SDR-SIM. Available online: <https://github.com/osqzss/gps-sdr-sim> (accessed on 12 May 2025).
9. Space Frequency Coordination Group. Communication and Positioning, Navigation, and Timing Frequency Allocations and Sharing in the Lunar Region; Recommendation SFCG REC 32-2R6. 18 June 2025. Available online: <https://www.sfcgonline.org/Resources/Recommendations/default.aspx> (accessed on 19 June 2025).
10. United States Space Force. IS-GPS-800, Revision J. 22 August 2022. Available online: <https://www.gps.gov/technical/icwg/IS-GPS-800J.pdf> (accessed on 12 May 2025).

11. Dafesh, P.A.; Khadge, G.K.; Wong, N.S.; Djuknic, G. Flexible Data and Frame Synchronization Structure for the LunaNet PNT Signal. In Proceedings of the ION 2024 Pacific PNT Meeting, Honolulu, HI, USA, 15–17 April 2024; pp. 844–866. [\[CrossRef\]](#)
12. European Union. Galileo Open Service Signal-in-Space Interface Control Document (OS SIS ICD), Issue 2.1. November 2023. Available online: [https://www.gsc-europa.eu/sites/default/files/sites/all/files/Galileo\\_OS\\_SIS\\_ICD\\_v2.1.pdf](https://www.gsc-europa.eu/sites/default/files/sites/all/files/Galileo_OS_SIS_ICD_v2.1.pdf) (accessed on 12 May 2025).
13. Sathaye, H.; Motallebighomi, M.; Ranganathan, A. Galileo-SDR-SIM: An Open-Source Tool for Generating Galileo Satellite Signals. In Proceedings of the 36th International Technical Meeting of the Satellite Division of the Institute of Navigation (ION GNSS+ 2023), Denver, CO, USA, 11–15 September 2023; pp. 3470–3480. [\[CrossRef\]](#)
14. GNSS-SDR-SIM. Available online: <https://github.com/Yuta811x/gnss-sdr-sim> (accessed on 12 May 2025).
15. Margana, B.S.; Achanta, D.S.; Songala, K.K.; Ammana, S.R. A Simple SDR based Method to Spoof Low-End GPS aided Drones for Securing Locations. In Proceedings of the 2021 IEEE International Conference on Robotics, Automation, Artificial-Intelligence and Internet-of-Things (RAAICON), Dhaka, Bangladesh, 29–30 December 2021; pp. 32–36. [\[CrossRef\]](#)
16. Ma, T.; Zhang, X.; Miao, Z. Detection of UAV GPS Spoofing Attacks Using a Stacked Ensemble Method. *Drones* **2025**, *9*, 2. [\[CrossRef\]](#)
17. GAL-OSNMA-SIM. Available online: <https://github.com/galileoos/gal-osnma-sim> (accessed on 12 May 2025).
18. Wiesmayr, R.; Nonaca, D.; Dick, C.; Studer, C. Optimizing Puncturing Patterns of 5G NR LDPC Codes for Few-Iteration Decoding. In Proceedings of the 58th Asilomar Conference on Signals, Systems, and Computers, Pacific Grove, CA, USA, 27–30 October 2024. [\[CrossRef\]](#)
19. LDPC-Codes. Available online: <https://github.com/radfordneal/LDPC-codes> (accessed on 12 May 2025).
20. OpenMP Architecture Review Board. OpenMP Application Programming Interface Examples, Version 6.0. November 2024. Available online: <https://www.openmp.org/wp-content/uploads/openmp-examples-6.0.pdf> (accessed on 12 May 2025).
21. RTKLIB. Available online: <https://github.com/tomojitakasu/RTKLIB> (accessed on 12 May 2025).
22. Ledvina, B.M.; Psiaki, M.L.; Powell, S.P.; Kintner, P.M. Bit-Wise Parallel Algorithms for Efficient Software Correlation Applied to a GPS Software Receiver. *IEEE Trans. Wirel. Commun.* **2004**, *3*, 1469–1473. [\[CrossRef\]](#)
23. Clements, Z.; Iannucci, P.A.; Humphreys, T.E.; Pany, T. Optimized Bit-Packing for Bit-Wise Software-Defined GNSS Radio. In Proceedings of the 34th International Technical Meeting of the Satellite Division of the Institute of Navigation (ION GNSS+ 2021), St. Louis, MO, USA, 20–24 September 2021; pp. 3749–3771. [\[CrossRef\]](#)
24. Nichols, H.A.; Murrian, M.J.; Humphreys, T.E. Software-Defined GNSS is Ready for Launch. In Proceedings of the 35th International Technical Meeting of the Satellite Division of the Institute of Navigation (ION GNSS+ 2022), Denver, CO, USA, 19–23 September 2022; pp. 996–1013. [\[CrossRef\]](#)
25. Spudis, P.D.; Bussey, B.; Plescia, J.; Josset, J.-L.; Beauvivre, S. Geology of Shackleton Crater and the south pole of the Moon. *Geophys. Res. Lett.* **2008**, *35*, 14. [\[CrossRef\]](#)
26. Murata, M.; Kawano, I.; Kogure, S. Lunar Navigation Satellite System and Positioning Accuracy Evaluation. In Proceedings of the 2022 International Technical Meeting of the Institute of Navigation, Long Beach, CA, USA, 25–27 January 2022; pp. 582–586. [\[CrossRef\]](#)
27. Murata, M.; Akiyama, K.; Satoh, N. Lunar South Pole Region Navigation Using Lunar Navigation Satellite System. In Proceedings of the 36th International Technical Meeting of the Satellite Division of the Institute of Navigation (ION GNSS+ 2023), Denver, CO, USA, 11–15 September 2023; pp. 3589–3596. [\[CrossRef\]](#)
28. Christopher, J.H. Analytical Model for GNSS Receiver Implementation Losses. *Navigation* **2011**, *58*, 29–44. [\[CrossRef\]](#)
29. bladeRF. Available online: <https://github.com/Nuand/bladeRF> (accessed on 12 May 2025).
30. Guertin, S.M. Raspberry Pi for Space Guidelines, NASA JPL. 2021. Available online: <https://nepp.nasa.gov/docs/papers/2021-Guertin-Raspberry-Pi-Guideline-CL-21-5641.pdf> (accessed on 12 May 2025).
31. Rabus, D.; Goavec-Merou, G.; Cabodevila, G.; Meyer, F.; Friedt, J.-M. Generating A Timing Information (1-PPS) From A Software Defined Radio Decoding of GPS Signals. In Proceedings of the 2021 Joint Conference of the European Frequency and Time Forum and IEEE International Frequency Control Symposium (EFTF/IFCS), Gainesville, FL, USA, 7–17 July 2021; pp. 1–2. [\[CrossRef\]](#)

**Disclaimer/Publisher’s Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.